Data Scalability and Analytics **BSIT - 3C**
Data transformation (Pyspark) **Oct 6, 2026**

Azul, Renz Dominic R.
de la Cruz, Gabriel R.
Galvez, Godwin Reigh M.

Repository Github Link:
https://github.com/User-Suuro/dsa-repo/tree/main/study_processed_data

Gdrive Link (with dataset):
https://drive.google.com/file/d/1dFgR9q4A_uNTL4-Adcj4A0w0olx8s4bv/view?usp=sharing

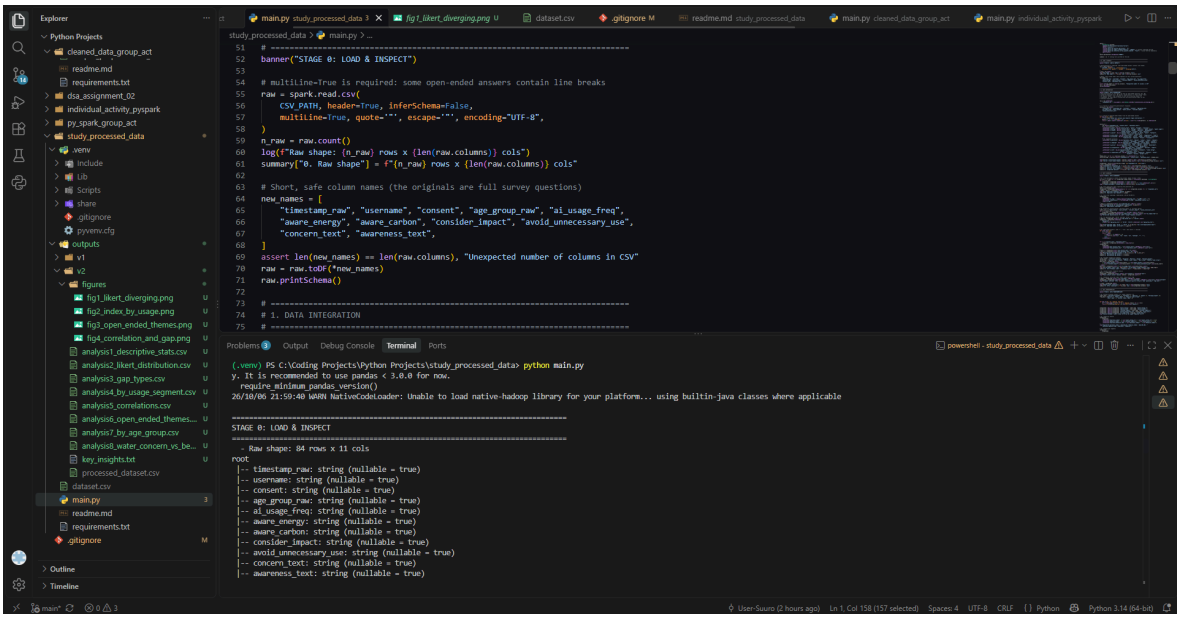
Dataset Size: 84 respondents, 11 initial survey features
Technologies Used: Python 3.11, Apache Spark (PySpark 3.5+), Pandas, Matplotlib, NumPy

Table of Member Contributions

Group Member Name	Role	Specific Contribution
Azul, Renz Dominic R.	Survey Designer & Researcher	Took part in brainstorming and selecting the research topic. Helped develop the objectives and research questions of the study. Designed and structured the Google Form questionnaire so that each item matched the research questions, including the multiple-choice and open-ended questions.
de la Cruz, Gabriel R.	Data Collection & Documentation	Took part in brainstorming and selecting the research topic. Conducted the survey through Google Forms and gathered the responses from the 84 respondents. Monitored the collected data for completeness and compiled the documents and requirements of the project.
Galvez, Godwin Reigh M.	Programmer	Participated in brainstorming and selecting the research topic. Developed Python code using Pandas and PySpark to inspect, clean, and prepare survey data, including converting open-ended responses (Q7 and Q8) into binary themes for analysis.

1. Dataset Inspection & Integration

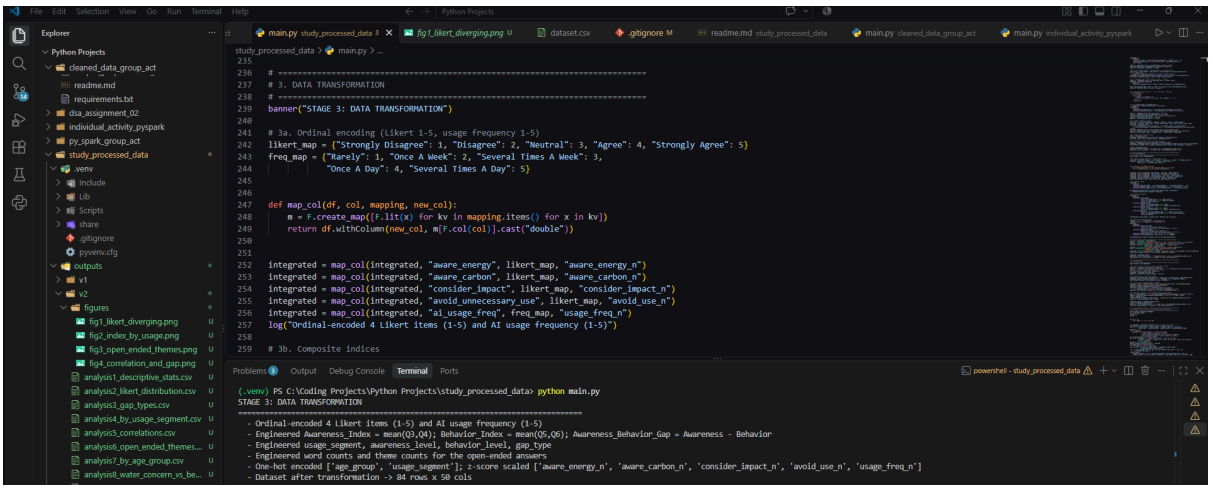
- **Raw Shape:** 84 rows × 11 columns.
- **Key Columns:** age_group_raw, ai_usage_freq, aware_energy (Q3), aware_carbon (Q4), consider_impact (Q5), avoid_unnecessary_use (Q6), concern_text (Q7), awareness_text (Q8).
- **Text Coding:** Regex used on open-ended answers (Q7, Q8) to extract key binary themes (c_water, c_energy, c_carbon, s_education, s_campaign, s_policy).



2. Data Cleaning & Transformations

- Whitespace & Nulls: Trimmed whitespace across all string columns; empty text set to NULL.
- Deduplication: Checked and removed duplicate submissions.
- Timestamp Parsing: Standardized timestamp strings to Spark TimestampType.
- Category Fixes: Corrected age label typo ('24-34' → '25-34').
- Metric Derivations:

Likert Scale: Strongly Disagree (1) to Strongly Agree (5)
Awareness Index = (aware_energy + aware_carbon) / 2
Behavior Index = (consider_impact + avoid_unnecessary_use) / 2
Awareness-Behavior Gap = Awareness Index - Behavior Index



3. PySpark DataFrame Analyses

Analysis 1: Overall Indices Summary

- Awareness Index: Mean = 3.85 / 5.0 (Agree range)
- Behavior Index: Mean = 3.81 / 5.0 (Agree range)
- Average Gap: +0.04 points (Strong balance between knowledge and action)

Analysis 2: Likert Response Distribution

- Highest Agreement: "Avoid unnecessary AI use" — 81.0% agreed.
- Lowest Agreement: "Consider env. impact when using AI" — 63.1% agreed (26.2% neutral).

Analysis 3: Awareness-to-Behavior Gap Typology

- Aligned (Gap within ±1.0): 76.2% (n = 64)
- Aware but Inactive (Gap greater than 1.0): 11.9% (n = 10)
- Action-Oriented (Gap less than -1.0): 11.9% (n = 10)

Analysis 4: Index Breakdown by AI Usage Frequency

Segment	Count (n)	Awareness Index	Behavior Index
Light Users (Rarely / Weekly)	7	4.29	4.36
Moderate Users (Several / Week)	32	4.09	4.00
Heavy Users (Daily / Multi-daily)	45	3.58	3.60

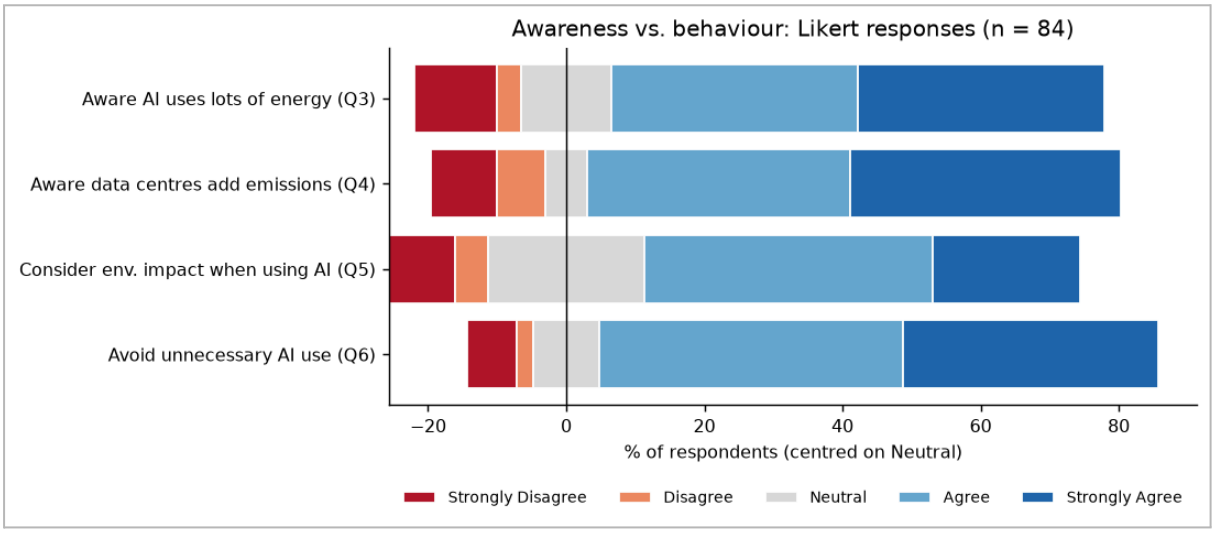
Finding: Heavy daily users exhibit lower awareness and conservation scores than light users.

Analysis 5: Correlation & Open-Ended Themes

- Correlation: Awareness vs. Behavior is strongly correlated (r = 0.777). Usage frequency vs. Behavior is weakly negative (r = -0.238).
- Top Concern (Q7): Water use / Cooling (55.7% of valid responses).
- Top Solution (Q8): Education & Training (51.9% of valid responses).

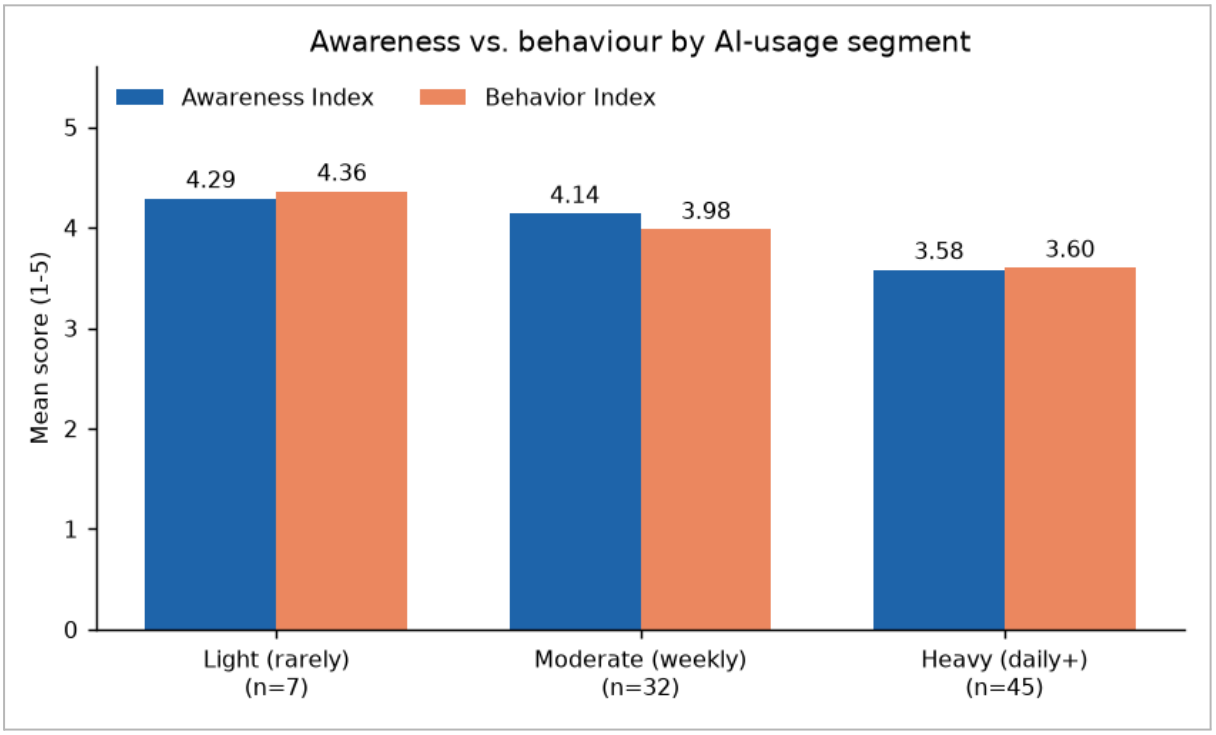
4. Visualizations & Interpretations

Figure 1: Likert Response Distribution



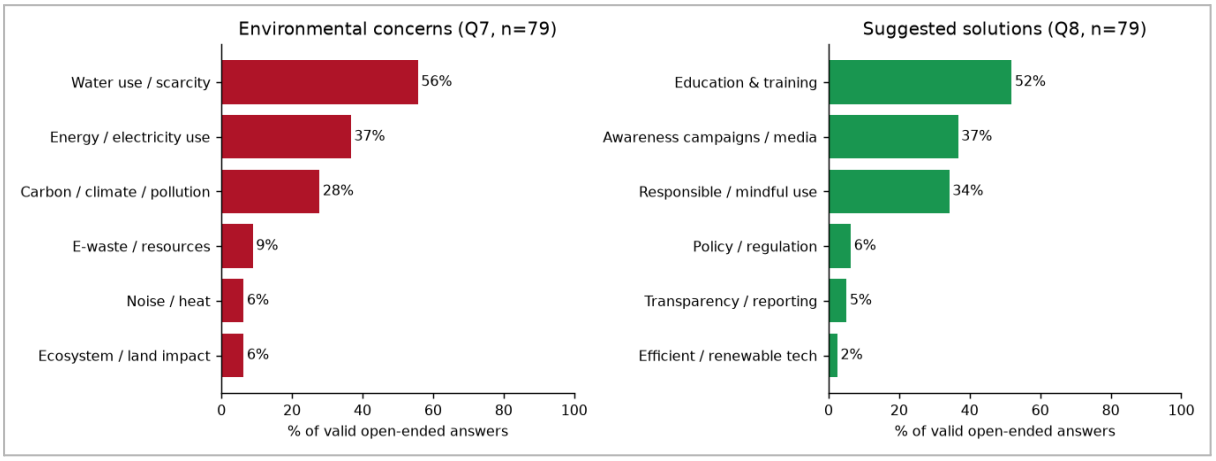
Interpretation: Shows strong overall agreement across all items, with real-time impact consideration (Q5) having the highest neutral/hesitant responses.

Figure 2: Awareness and Behavior by Usage Segment



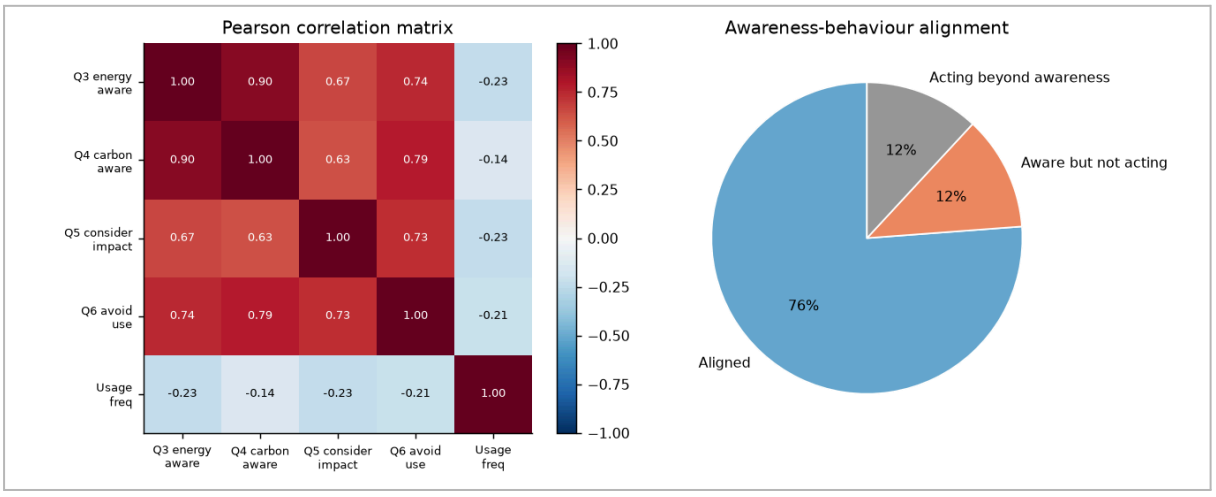
Interpretation: Clearly illustrates that heavy daily AI users have lower average sustainability scores compared to light users.

Figure 3: Qualitative Themes (Concerns vs. Solutions)



Interpretation: Confirms data center water cooling as the primary environmental concern and institutional education as the preferred remedy.

Figure 4: Correlation & Gap Distribution

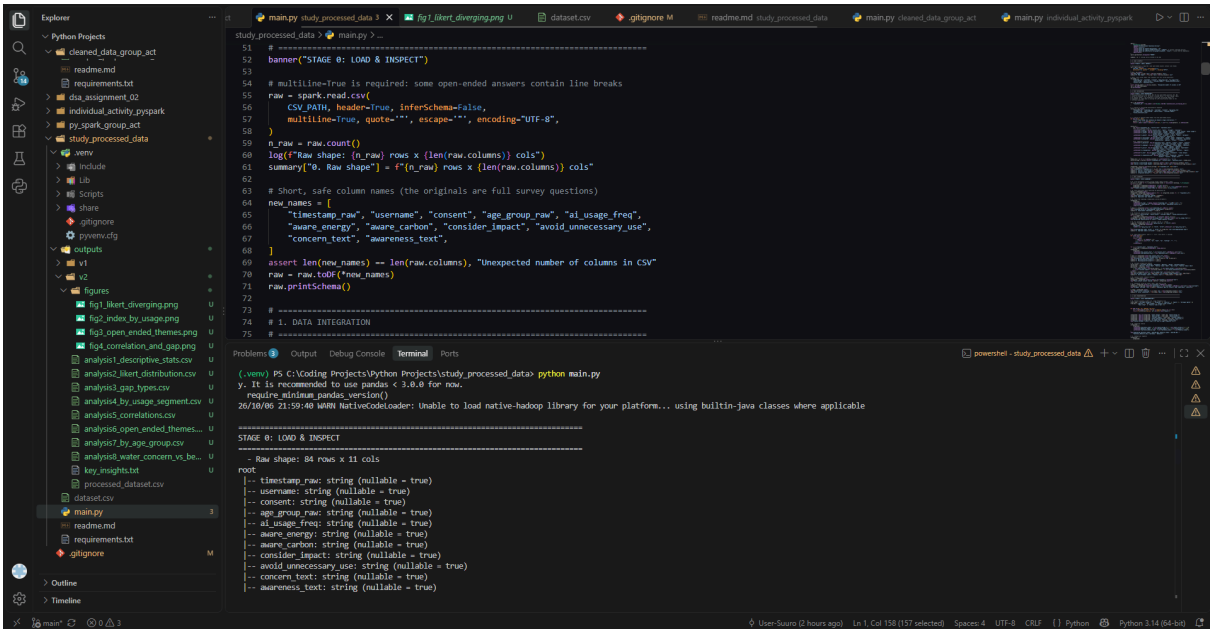


Interpretation: Confirms a strong linear relationship (r = 0.777) between awareness and behavior, with 76.2% of users categorized as aligned.

5. Conclusion & Recommendations

- Target Heavy Users: Introduce eco-prompts or usage summaries for frequent AI users.
- Water Footprint Awareness: Expand educational campaigns to cover data center water consumption.
- Curriculum Integration: Integrate green computing principles into academic AI programs.

6. PySpark Source Code Snippet



```
# ============================================================================
banner("STAGE 0: LOAD & INSPECT")

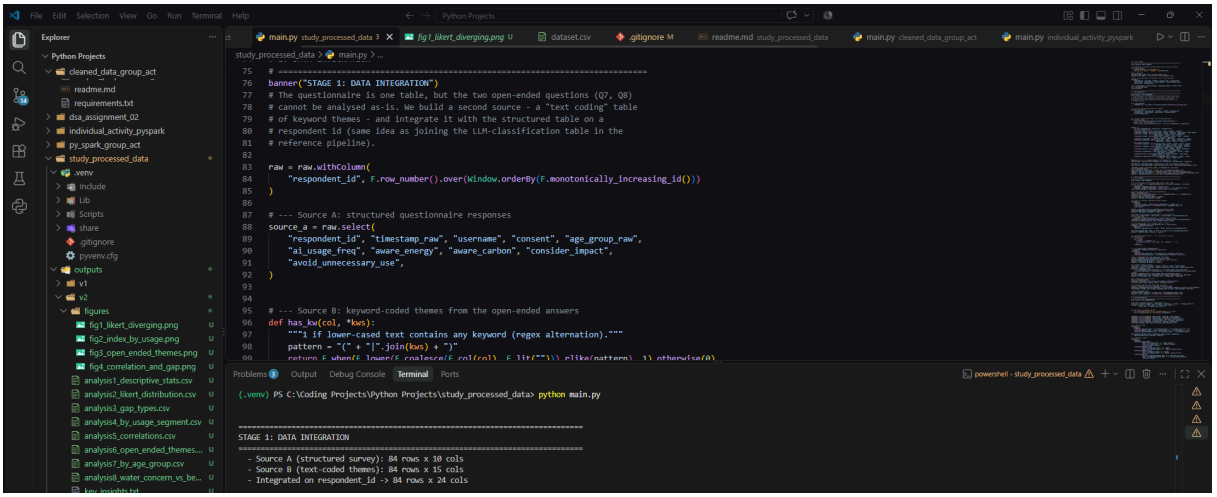
# multiLine=True is required: some open-ended answers contain line breaks
raw = spark.read.csv(
    CSV_PATH, header=True, inferSchema=False,
    multiLine=True, quote='"', escape='"', encoding="UTF-8",
)
n_raw = raw.count()
log(f"Raw shape: {n_raw} rows x {len(raw.columns)} cols")
summary["0. Raw shape"] = f"{n_raw} rows x {len(raw.columns)} cols"

# Short, safe column names (the originals are full survey questions)
new_names = [
    "timestamp_raw", "username", "consent", "age_group_raw", "ai_usage_freq",
    "aware_energy", "aware_carbon", "consider_impact", "avoid_unnecessary_use",
    "concern_text", "awareness_text",
]
assert len(new_names) == len(raw.columns), "Unexpected number of columns in CSV"
raw = raw.toDF(*new_names)
raw.printSchema()

# ============================================================================
# 1. DATA INTEGRATION
# ============================================================================
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py
y. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
26/10/06 21:59:40 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

================================================================================
STAGE 0: LOAD & INSPECT
================================================================================
  - Raw shape: 84 rows x 11 cols
root
 |-- timestamp_raw: string (nullable = true)
 |-- username: string (nullable = true)
 |-- consent: string (nullable = true)
 |-- age_group_raw: string (nullable = true)
 |-- ai_usage_freq: string (nullable = true)
 |-- aware_energy: string (nullable = true)
 |-- aware_carbon: string (nullable = true)
 |-- consider_impact: string (nullable = true)
 |-- avoid_unnecessary_use: string (nullable = true)
 |-- concern_text: string (nullable = true)
 |-- awareness_text: string (nullable = true)
```



```
# ============================================================================
banner("STAGE 1: DATA INTEGRATION")
# The questionnaire is one table, but the two open-ended questions (Q7, Q8)
# cannot be analysed as-is. We build a second source - a "text coding" table
# of keyword themes - and integrate it with the structured table on a
# respondent id (same idea as joining the LLM-classification table in the
# reference pipeline).

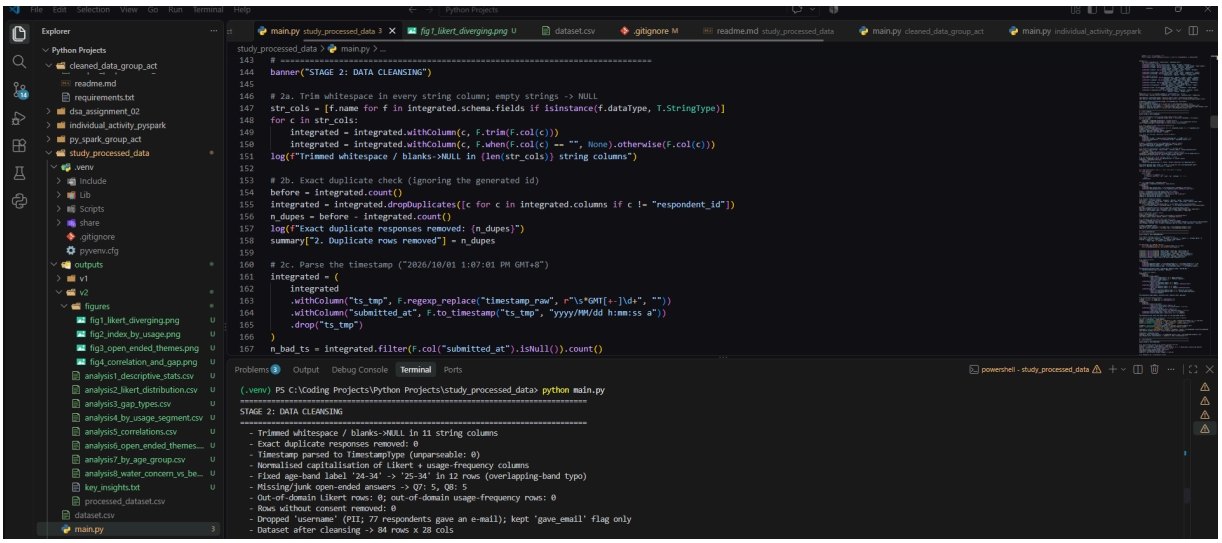
raw = raw.withColumn(
    "respondent_id", F.row_number().over(Window.orderBy(F.monotonically_increasing_id()))
)

# --- Source A: structured questionnaire responses
source_a = raw.select(
    "respondent_id", "timestamp_raw", "username", "consent", "age_group_raw",
    "ai_usage_freq", "aware_energy", "aware_carbon", "consider_impact",
    "avoid_unnecessary_use",
)


# --- Source B: keyword-coded themes from the open-ended answers
def has_kw(col, *kws):
    """1 if lower-cased text contains any keyword (regex alternation)."""
    pattern = "(" + "|".join(kws) + ")"
    return F.when(F.lower(F.coalesce(F.col(col), F.lit(""))).rlike(pattern), 1).otherwise(0)
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py


================================================================================
STAGE 1: DATA INTEGRATION
================================================================================
  - Source A (structured survey): 84 rows x 10 cols
  - Source B (text-coded themes): 84 rows x 15 cols
  - Integrated on respondent_id -> 84 rows x 24 cols
```



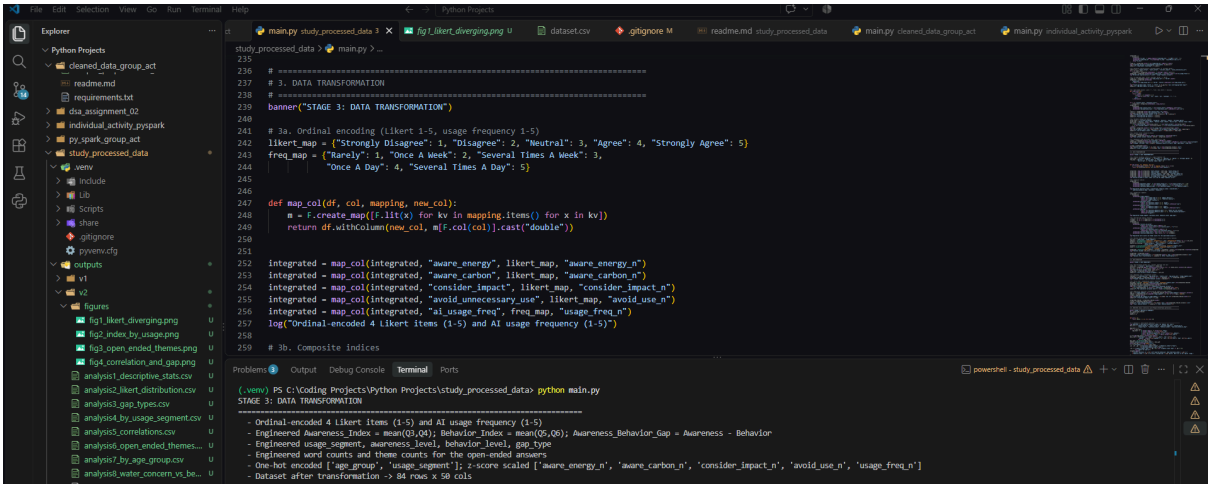
```
# ============================================================================
banner("STAGE 2: DATA CLEANSING")

# 2a. Trim whitespace in every string column; empty strings -> NULL
str_cols = [f.name for f in integrated.schema.fields if isinstance(f.dataType, T.StringType)]
for c in str_cols:
    integrated = integrated.withColumn(c, F.trim(F.col(c)))
    integrated = integrated.withColumn(c, F.when(F.col(c) == "", None).otherwise(F.col(c)))
log(f"Trimmed whitespace / blanks->NULL in {len(str_cols)} string columns")

# 2b. Exact duplicate check (ignoring the generated id)
before = integrated.count()
integrated = integrated.dropDuplicates([c for c in integrated.columns if c != "respondent_id"])
n_dupes = before - integrated.count()
log(f"Exact duplicate responses removed: {n_dupes}")
summary["2. Duplicate rows removed"] = n_dupes

# 2c. Parse the timestamp ("2026/10/01 1:07:01 PM GMT+8")
integrated = (
    integrated
    .withColumn("ts_tmp", F.regexp_replace("timestamp_raw", r"\s*GMT[+-]\d+", ""))
    .withColumn("submitted_at", F.to_timestamp("ts_tmp", "yyyy/MM/dd h:mm:ss a"))
    .drop("ts_tmp")
)
n_bad_ts = integrated.filter(F.col("submitted_at").isNull()).count()
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py
================================================================================
STAGE 2: DATA CLEANSING
================================================================================
  - Trimmed whitespace / blanks->NULL in 11 string columns
  - Exact duplicate responses removed: 0
  - Timestamp parsed to TimestampType (unparseable: 0)
  - Normalised capitalisation of Likert + usage-frequency columns
  - Fixed age-band label '24-34' -> '25-34' in 12 rows (overlapping-band typo)
  - Missing/junk open-ended answers -> Q7: 5, Q8: 5
  - Out-of-domain Likert rows: 0; out-of-domain usage-frequency rows: 0
  - Rows without consent removed: 0
  - Dropped 'username' (PII; 77 respondents gave an e-mail); kept 'gave_email' flag only
  - Dataset after cleansing -> 84 rows x 28 cols
```



```
# ============================================================================
# 3. DATA TRANSFORMATION
# ============================================================================
banner("STAGE 3: DATA TRANSFORMATION")

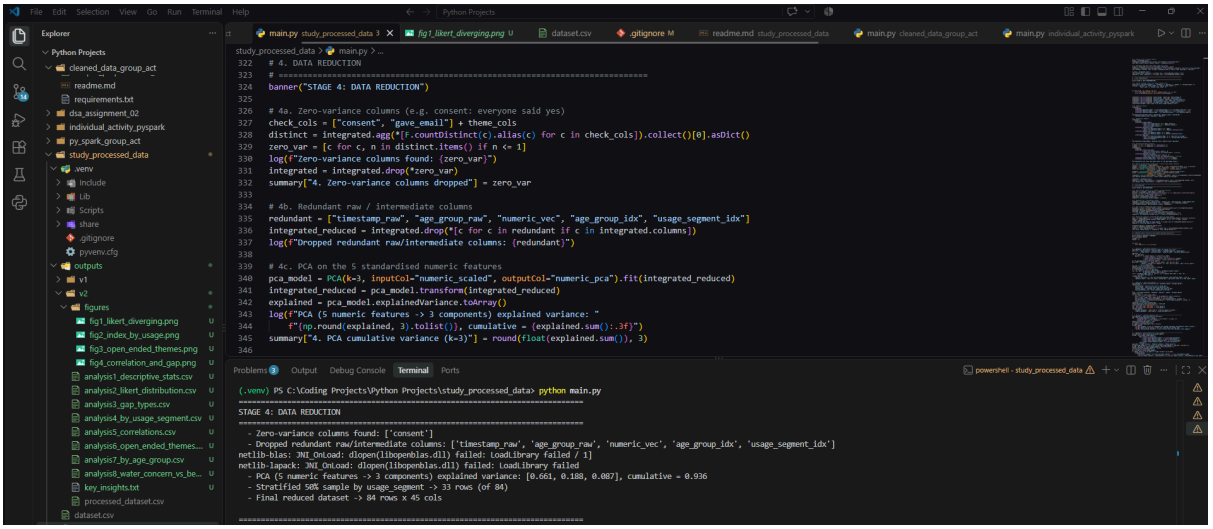
# 3a. Ordinal encoding (Likert 1-5, usage frequency 1-5)
likert_map = {"Strongly Disagree": 1, "Disagree": 2, "Neutral": 3, "Agree": 4, "Strongly Agree": 5}
freq_map = {"Rarely": 1, "Once A Week": 2, "Several Times A Week": 3,
            "Once A Day": 4, "Several Times A Day": 5}


def map_col(df, col, mapping, new_col):
    m = F.create_map([F.lit(x) for kv in mapping.items() for x in kv])
    return df.withColumn(new_col, m[F.col(col)].cast("double"))


integrated = map_col(integrated, "aware_energy", likert_map, "aware_energy_n")
integrated = map_col(integrated, "aware_carbon", likert_map, "aware_carbon_n")
integrated = map_col(integrated, "consider_impact", likert_map, "consider_impact_n")
integrated = map_col(integrated, "avoid_unnecessary_use", likert_map, "avoid_use_n")
integrated = map_col(integrated, "ai_usage_freq", freq_map, "usage_freq_n")
log("Ordinal-encoded 4 Likert items (1-5) and AI usage frequency (1-5)")

# 3b. Composite indices
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py
STAGE 3: DATA TRANSFORMATION
================================================================================
  - Ordinal-encoded 4 Likert items (1-5) and AI usage frequency (1-5)
  - Engineered Awareness_Index = mean(Q3,Q4); Behavior_Index = mean(Q5,Q6); Awareness_Behavior_Gap = Awareness - Behavior
  - Engineered usage_segment, awareness_level, behavior_level, gap_type
  - Engineered word counts and theme counts for the open-ended answers
  - One-hot encoded ['age_group', 'usage_segment']; z-score scaled ['aware_energy_n', 'aware_carbon_n', 'consider_impact_n', 'avoid_use_n', 'usage_freq_n']
  - Dataset after transformation -> 84 rows x 50 cols
```



```
# ============================================================================
# 4. DATA REDUCTION
# ============================================================================
banner("STAGE 4: DATA REDUCTION")

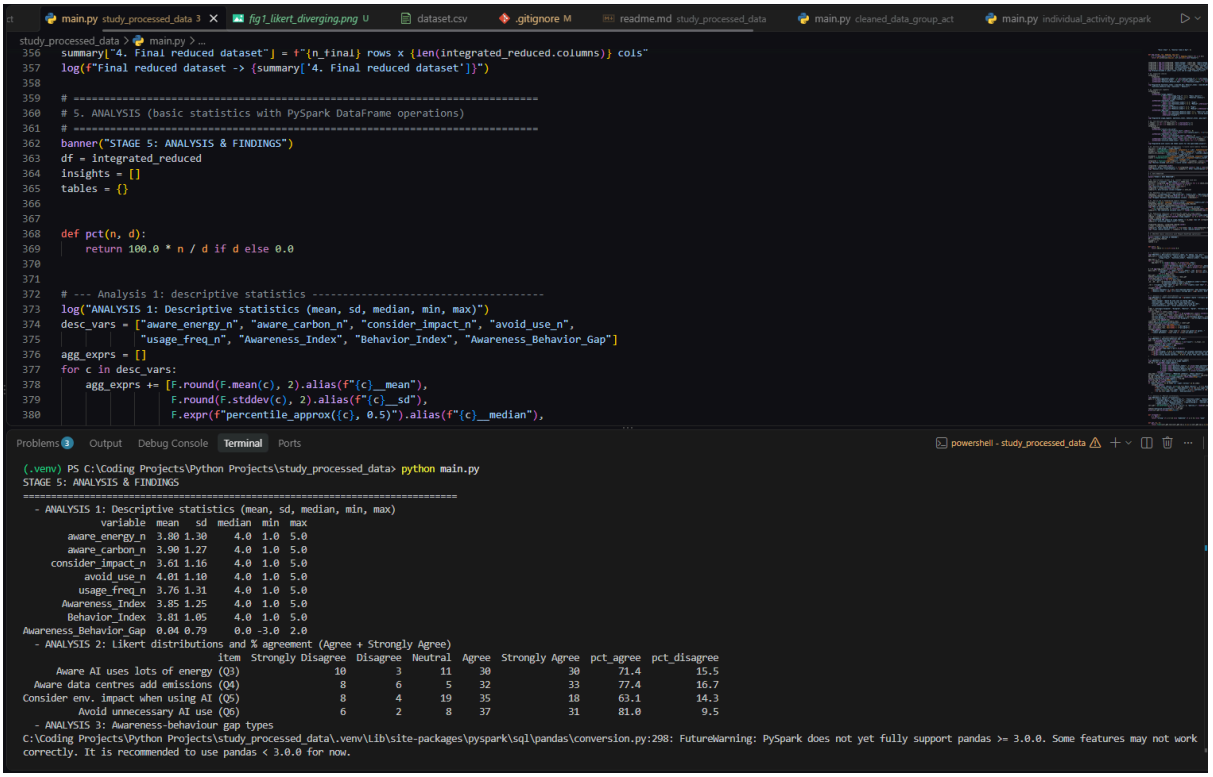
# 4a. Zero-variance columns (e.g. consent: everyone said yes)
check_cols = ["consent", "gave_email"] + theme_cols
distinct = integrated.agg(*[F.countDistinct(c).alias(c) for c in check_cols]).collect()[0].asDict()
zero_var = [c for c, n in distinct.items() if n <= 1]
log(f"Zero-variance columns found: {zero_var}")
integrated = integrated.drop(*zero_var)
summary["4. Zero-variance columns dropped"] = zero_var

# 4b. Redundant raw / intermediate columns
redundant = ["timestamp_raw", "age_group_raw", "numeric_vec", "age_group_idx", "usage_segment_idx"]
integrated_reduced = integrated.drop(*[c for c in redundant if c in integrated.columns])
log(f"Dropped redundant raw/intermediate columns: {redundant}")

# 4c. PCA on the 5 standardised numeric features
pca_model = PCA(k=3, inputCol="numeric_scaled", outputCol="numeric_pca").fit(integrated_reduced)
integrated_reduced = pca_model.transform(integrated_reduced)
explained = pca_model.explainedVariance.toArray()
log(f"PCA (5 numeric features -> 3 components) explained variance: "
    f"{np.round(explained, 3).tolist()}, cumulative = {explained.sum():.3f}")
summary["4. PCA cumulative variance (k=3)"] = round(float(explained.sum()), 3)
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py
================================================================================
STAGE 4: DATA REDUCTION
================================================================================
  - Zero-variance columns found: ['consent']
  - Dropped redundant raw/intermediate columns: ['timestamp_raw', 'age_group_raw', 'numeric_vec', 'age_group_idx', 'usage_segment_idx']
netlib-blas: JNI_OnLoad: dlopen(libopenblas.dll) failed: LoadLibrary failed / 1]
netlib-lapack: JNI_OnLoad: dlopen(libopenblas.dll) failed: LoadLibrary failed
  - PCA (5 numeric features -> 3 components) explained variance: [0.661, 0.188, 0.087], cumulative = 0.936
  - Stratified 50% sample by usage_segment -> 33 rows (of 84)
  - Final reduced dataset -> 84 rows x 45 cols

================================================================================
```



```
    summary["4. Final reduced dataset"] = f"{n_final} rows x {len(integrated_reduced.columns)} cols"
    log(f"Final reduced dataset -> {summary['4. Final reduced dataset']}")

    # ============================================================================
    # 5. ANALYSIS (basic statistics with PySpark DataFrame operations)
    # ============================================================================
    banner("STAGE 5: ANALYSIS & FINDINGS")
    df = integrated_reduced
    insights = []
    tables = {}


    def pct(n, d):
        return 100.0 * n / d if d else 0.0


    # --- Analysis 1: descriptive statistics ------------------------------------
    log("ANALYSIS 1: Descriptive statistics (mean, sd, median, min, max)")
    desc_vars = ["aware_energy_n", "aware_carbon_n", "consider_impact_n", "avoid_use_n",
                 "usage_freq_n", "Awareness_Index", "Behavior_Index", "Awareness_Behavior_Gap"]
    agg_exprs = []
    for c in desc_vars:
        agg_exprs += [F.round(F.mean(c), 2).alias(f"{c}__mean"),
                      F.round(F.stddev(c), 2).alias(f"{c}__sd"),
                      F.expr(f"percentile_approx({c}, 0.5)").alias(f"{c}__median"),
```

```
(.venv) PS C:\Coding Projects\Python Projects\study_processed_data> python main.py
STAGE 5: ANALYSIS & FINDINGS
================================================================================
  - ANALYSIS 1: Descriptive statistics (mean, sd, median, min, max)
            variable  mean   sd  median  min  max
       aware_energy_n  3.80 1.30     4.0  1.0  5.0
       aware_carbon_n  3.90 1.27     4.0  1.0  5.0
    consider_impact_n  3.61 1.16     4.0  1.0  5.0
         avoid_use_n  4.01 1.10     4.0  1.0  5.0
        usage_freq_n  3.76 1.31     4.0  1.0  5.0
    Awareness_Index  3.85 1.25     4.0  1.0  5.0
     Behavior_Index  3.81 1.05     4.0  1.0  5.0
Awareness_Behavior_Gap  0.04 0.79     0.0 -3.0  2.0
  - ANALYSIS 2: Likert distributions and % agreement (Agree + Strongly Agree)
                      item  Strongly Disagree  Disagree  Neutral  Agree  Strongly Agree  pct_agree  pct_disagree
    Aware AI uses lots of energy (Q3)                 10         3       11     30              30       71.4          15.5
 Aware data centres add emissions (Q4)                 8         6        5     32              33       77.4          16.7
Consider env. impact when using AI (Q5)                8         4       19     35              18       63.1          14.3
        Avoid unnecessary AI use (Q6)                  6         2        8     37              31       81.0           9.5
  - ANALYSIS 3: Awareness-behaviour gap types
C:\Coding Projects\Python Projects\study_processed_data\.venv\Lib\site-packages\pyspark\sql\pandas\conversion.py:298: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work
correctly. It is recommended to use pandas < 3.0.0 for now.
```